

Nmap – Tipos de Escaneo

Guia Detallada para Hacking y Pentesting

Escaneos cubiertos:

- sU UDP Scan
- Pn Skip Host Discovery
- sn Ping Scan
- sA ACK Scan
- sC Default Scripts
- sN NULL Scan
- sF FIN Scan
- sX Xmas Scan
- s0 Protocol Scan

Contents

1. -sU – UDP Scan

1.1. Que es

El escaneo UDP envia paquetes UDP a los puertos objetivo y analiza las respuestas para determinar si estan abiertos o cerrados. A diferencia de TCP, UDP no tiene handshake, lo que hace la deteccion mas compleja.

1.2. Como funciona

UDP no tiene mecanismo de confirmacion como el three-way handshake de TCP. Nmap envia un paquete UDP (vacio o con payload especifico segun el servicio) y espera la respuesta:

Respuesta	Estado del puerto
No responde	open filtered
Responde ICMP port unreachable	closed
Responde con UDP	open

1.3. Que pasa si mandas UDP a un puerto TCP como el 22

El puerto 22 pertenece a SSH que es un protocolo TCP. Si mandas un paquete UDP al puerto 22 esto es lo que ocurre:

1. El sistema operativo recibe el paquete UDP en el puerto 22
2. Ningun proceso esta escuchando UDP en ese puerto (SSH escucha solo TCP)
3. El kernel responde automaticamente con **ICMP port unreachable**
4. Nmap lo marca como **closed**

UDP y TCP son protocolos completamente separados. El OS los gestiona de forma independiente. Un servicio TCP no puede recibir trafico UDP aunque compartan el mismo numero de puerto.

1.4. Por que es lento

Linux limita las respuestas ICMP a 1 por segundo por defecto (RFC 1812). Si escaneas 65535 puertos UDP en el peor caso tardaria aproximadamente 18 horas. Por eso se recomienda limitar el escaneo a los puertos UDP mas comunes.

1.5. Servicios UDP mas comunes

Puerto	Servicio	Uso
53	DNS	Resolucion de nombres
67	DHCP	Asignacion de IPs
69	TFTP	Transferencia de archivos simple
123	NTP	Sincronizacion de tiempo
161	SNMP	Gestion de dispositivos de red
500	IKE	VPN IPSec

1.6. Uso

```
# escaneo UDP puertos comunes
nmap -sU --top-ports 100 192.168.1.1

# combinado TCP y UDP
nmap -sS -sU -p T:80,443,22,U:53,161 192.168.1.1

# escaneo UDP completo (muy lento)
nmap -sU -p- 192.168.1.1
```

2. -Pn – Skip Host Discovery

2.1. Que es

Por defecto nmap realiza un host discovery antes de escanear puertos para verificar que el host esta activo. -Pn desactiva ese paso y asume que el host esta vivo sin comprobarlo.

2.2. El proceso normal sin -Pn

1. nmap envia ICMP echo request (ping)
2. si no responde -> asume host muerto -> no escanea
3. si responde -> procede a escanear puertos

2.3. Por que usarlo

Muchos sistemas bloquean ICMP (ping) por politica de seguridad:

Situacion	Por que -Pn
Firewalls que bloquean ICMP	El host esta vivo pero no responde ping
Windows por defecto	Bloquea ICMP echo request
Servidores cloud	AWS, Azure y GCP suelen bloquear ICMP
Hosts con IDS	Pueden ignorar ICMP deliberadamente

Sin -Pn, nmap pensaria que el host esta muerto y no escanearia ningun puerto, dando un falso negativo.

2.4. Uso

```
# escanear aunque no responda ping
nmap -sS -Pn 192.168.1.1

# combinado con otros parametros
nmap -sS -sV -Pn -n -p- 192.168.1.1
```

3. -sn – Ping Scan

3.1. Que es

-sn realiza solo host discovery sin escanear puertos. Su objetivo es determinar que hosts estan activos en una red o IP.

3.2. Como funciona segun el contexto

El comportamiento cambia dependiendo de si escaneas una red local o un servidor remoto:

3.2.1. En red local (LAN)

```
usa ARP requests (capa 2)
ARP es mas confiable porque opera por debajo de IP
es muy dificil bloquear ARP en una red local
resultado: muy preciso en redes locales
```

3.2.2. En servidor remoto (internet)

```
usa ICMP echo request (ping)
TCP SYN al puerto 443
TCP ACK al puerto 80
si el servidor bloquea todo esto -> nmap dice muerto aunque este vivo
```

3.3. Que pasa con un servidor al otro lado del mundo

Para un servidor remoto -sn es menos confiable porque:

1. Hay mas probabilidad de que ICMP este bloqueado por firewall
2. Los puertos 80 y 443 pueden estar filtrados
3. Un falso negativo es mas probable

En ese caso se combina con -Pn para forzar el escaneo de puertos directamente sin depender del host discovery.

3.4. Uso

```
# descubrir hosts activos en toda la red
nmap -sn 192.168.1.0/24

# descubrir hosts en un rango
nmap -sn 192.168.1.1-254

# descubrir host especifico
nmap -sn 192.168.1.1
```

4. -sA – ACK Scan

4.1. Que es

El ACK scan no detecta puertos abiertos. Su funcion es detectar la presencia y comportamiento de firewalls, mapeando que puertos estan filtrados y cuales no.

4.2. Como funciona

Nmap envia un paquete TCP con el flag ACK activado sin que haya habido una conexion previa (sin SYN previo):

Respuesta	Interpretacion
Recibe RST	Puerto unfiltered (sin firewall bloqueando)
No responde	Puerto filtered (firewall bloqueando)

4.3. Por que un ACK sin SYN revela el firewall

En una conexion TCP normal el flujo es:

```
cliente -> SYN          ->  servidor
cliente <- SYN/ACK      <-  servidor
cliente -> ACK          ->  servidor
```

Si el cliente manda un ACK sin haber enviado SYN primero:

- **Host sin firewall:** el kernel recibe un ACK inesperado y responde RST porque no existe esa conexion. Puerto **unfiltered**.
- **Host con firewall stateful:** el firewall descarta el paquete porque no hay estado de conexion previo. Puerto **filtered**.
- **Host con firewall stateless:** depende de las reglas configuradas.

4.4. Que informacion da y para que sirve

1. **Mapear reglas del firewall:** saber exactamente que puertos filtra y cuales no
2. **Distinguir firewall stateful de stateless:** el comportamiento es diferente
3. **Preparar evasion:** conocer los puertos no filtrados para usarlos como vector
4. **Confirmar arquitectura de red:** entender como esta estructurada la defensa

Importante: -sA no dice si un puerto esta abierto o cerrado. Solo dice si hay un firewall entre tu y el host. Para detectar puertos abiertos usa -sS o -sT.

4.5. Uso

```
# detectar firewall
nmap -sA 192.168.1.1

# en puertos especificos
nmap -sA -p 80,443,22,8080 192.168.1.1
```


5. -sC – Default Scripts

5.1. Que es

-sC ejecuta los scripts de la categoria default del NSE (Nmap Scripting Engine). Son scripts considerados seguros y utiles que se ejecutan automaticamente segun el servicio detectado en cada puerto.

5.2. Scripts que ejecuta segun el servicio

5.2.1. SSH – Puerto 22

```
ssh-hostkey      -> muestra las claves publicas del servidor SSH  
                  (RSA, ECDSA, ED25519)
```

5.2.2. HTTP/HTTPS – Puertos 80 y 443

```
http-title       -> titulo de la pagina web  
http-server-header -> version del servidor web  
http-auth-finder -> detecta metodos de autenticacion
```

5.2.3. FTP – Puerto 21

```
ftp-anon        -> detecta si permite login anonimo sin contrasena  
ftp-bounce      -> detecta si permite ataques de rebote FTP
```

5.2.4. SMB – Puerto 445

```
smb-security-mode -> configuracion de seguridad SMB  
smb-os-discovery  -> detecta sistema operativo via SMB
```

5.2.5. MySQL – Puerto 3306

```
mysql-info       -> version, capabilities del servidor MySQL  
mysql-empty-password -> detecta acceso sin contrasena
```

5.2.6. SMTP – Puerto 25

```
smtp-commands    -> comandos disponibles en el servidor de email  
smtp-open-relay  -> detecta si permite relay abierto
```

5.3. Output tipico

```
22/tcp open  ssh  
| ssh-hostkey:  
|   2048 ab:cd:ef:12 (RSA)  
|_   256 34:56:78:90 (ECDSA)
```

```
80/tcp open  http
|_http-title: Apache2 Ubuntu Default Page
|_http-server-header: Apache/2.4.41

21/tcp open  ftp
|_ftp-anon: Anonymous FTP login allowed
```

5.4. Uso

```
# ejecutar scripts por defecto
nmap -sC 192.168.1.1

# combinado con deteccion de versiones
nmap -sC -sV 192.168.1.1

# equivalente a -sC -sV -O
nmap -A 192.168.1.1
```

6. -sN, -sF y -sX – Escaneos Sigilosos

6.1. La base teorica: RFC 793

Estos tres escaneos se basan en el comportamiento definido en el RFC 793, la especificacion original del protocolo TCP:

RFC 793 establece:

Si un puerto **cerrado** recibe un paquete TCP sin los flags SYN, ACK o RST activos, debe responder con RST.

Si un puerto **abierto** recibe un paquete TCP sin los flags SYN, ACK o RST activos, debe ignorarlo silenciosamente.

Nmap explota este comportamiento para detectar puertos sin necesidad de enviar SYN (que es el metodo mas detectado por firewalls e IDS).

6.2. -sN – NULL Scan

Envia un paquete TCP sin ningun flag activado. Es literalmente un paquete vacio a nivel de flags.

```
paquete enviado: TCP sin flags (todos en 0)

puerto abierto  -> silencio (no responde)
puerto cerrado  -> RST
puerto filtrado -> silencio o ICMP unreachable
```

6.3. -sF – FIN Scan

Envia un paquete TCP con solo el flag FIN activado. FIN normalmente se usa para cerrar conexiones establecidas.

```
paquete enviado: TCP FIN

puerto abierto  -> silencio
puerto cerrado  -> RST
puerto filtrado -> silencio
```

6.4. -sX – Xmas Scan

Envia un paquete TCP con tres flags activados simultaneamente: FIN + PSH + URG. Se llama Xmas porque en Wireshark el paquete aparece con muchos bits encendidos, como un arbol de navidad iluminado.

```
paquete enviado: TCP FIN + PSH + URG

puerto abierto  -> silencio
puerto cerrado  -> RST
puerto filtrado -> silencio
```

6.5. Por que evaden firewalls

Muchos firewalls simples y stateless aplican reglas basadas en el flag SYN. La logica es: si no hay SYN, no hay inicio de conexion, no hay que inspeccionar. Un paquete NULL, FIN o Xmas puede pasar sin ser inspeccionado porque no tiene SYN.

6.6. El problema critico con Windows

Limitacion importante:

Windows **no sigue** el RFC 793 en este aspecto. Responde RST tanto en puertos abiertos como en cerrados al recibir estos paquetes.

Resultado: estos tres escaneos no distinguen puertos abiertos de cerrados en Windows. Funcionan correctamente en Linux, Unix, macOS y la mayoría de sistemas tipo Unix.

6.7. Comparacion

Escaneo	Flags enviados	Linux abierto	Windows abierto
-sN	Ninguno	Silencio	RST
-sF	FIN	Silencio	RST
-sX	FIN+PSH+URG	Silencio	RST

6.8. Uso

```
nmap -sN 192.168.1.1    # NULL scan
nmap -sF 192.168.1.1    # FIN scan
nmap -sX 192.168.1.1    # Xmas scan

# con velocidad reducida para mayor sigilo
nmap -sX -T1 192.168.1.1
```

7. -sO – Protocol Scan

7.1. Que es

-sO no escanea puertos TCP o UDP. Escanea que protocolos IP soporta el host a nivel de capa 3 del modelo OSI, modificando el campo *protocol* en el header IP.

7.2. Como funciona

Nmap envia paquetes IP con diferentes numeros de protocolo en el header y analiza las respuestas:

Respuesta	Estado
Cualquier respuesta o silencio	Protocolo open filtered
ICMP protocol unreachable	Protocolo closed (no soportado)

7.3. Protocolos que detecta

Numero	Protocolo	Uso
1	ICMP	Mensajes de control (ping)
6	TCP	Transporte confiable
17	UDP	Transporte sin conexion
47	GRE	Tunneling (VPN PPTP)
50	ESP	IPSec cifrado
51	AH	IPSec autenticacion
89	OSPF	Protocolo de enrutamiento

7.4. Para que sirve en hacking

1. **Detectar VPN activa:** si el host soporta GRE, ESP o AH probablemente tiene VPN configurada
2. **Entender arquitectura de red:** que protocolos acepta revela como esta configurada la red
3. **Encontrar protocolos menos monitoreados:** para tunelizar trafico por protocolos que el IDS no inspecciona tan agresivamente
4. **Evasion avanzada:** si ICMP esta bloqueado pero GRE no, se puede usar GRE como canal de comunicacion

7.5. Uso

```
# escaneo de protocolos IP
nmap -sO 192.168.1.1

# combinado con -v para mas detalle
nmap -sO -v 192.168.1.1
```

8. Combinaciones utiles en hacking

```
# escaneo completo estandar
nmap -sS -sV -sC -O -p- -T4 192.168.1.1

# cuando el host no responde ping
nmap -sS -Pn -sV 192.168.1.1

# descubrir hosts en la red
nmap -sn 192.168.1.0/24

# sigiloso y completo
nmap -sS -sV -O -Pn -n -T2 -p- 192.168.1.1

# mapear firewall antes de atacar
nmap -sA -p 80,443,22,8080,3306 192.168.1.1

# escaneo UDP de servicios criticos
nmap -sU --top-ports 100 -T4 192.168.1.1

# buscar vulnerabilidades
nmap -sV --script=vuln 192.168.1.1

# evasion con escaneo sigiloso
nmap -sX -T1 -n -Pn 192.168.1.1
```

Regla de oro: siempre agregar 2>/dev/null cuando se usa nmap en scripts para descartar errores de permisos y obtener solo los resultados utiles.